归位 GuiWei 开发日志

从选题到模拟器跑通的完整记录

对应题目：题目一 物品管理 App（原生客户端）

环境：Xcode 27.0 · Swift 6.4 · iOS 27 SDK · iPhone 17 Pro 模拟器

本日志按实际开发顺序记录，包含全部踩坑过程与验证命令

目 录

第 0 章 这份日志记录了什么 5

0.1 一句话结论 5

0.2 这份日志的写法 5

0.3 成品长什么样 5

第 1 章 选题：为什么是「归位」 8

1.1 题目给了什么约束 8

1.2 三个候选方向 8

1.3 最终定位 9

1.4 AirTag 的技术真相（这一节很重要） 9

第 2 章 环境确认与工程搭建 11

2.1 先确认环境，再动手 11

2.2 手工编写工程文件 11

2.3 第一个障碍：构建目录在工程外面 12

2.4 第二个障碍：宏插件的沙盒（本节是整份日志里最有价值的一段） 13

2.5 唯一的一个真实编译错误 14

2.6 图标与配色 14

第 3 章 数据模型设计 16

3.1 四个模型 16

3.2 位置为什么用自引用树 16

3.3 图片的两级存储 17

3.4 追踪标签为什么要抽象成值类型 17

3.5 派生状态用计算属性，不落库 18

3.6 数据真的落盘了吗（实证） 18

第 4 章 服务层：把"和系统打交道"的部分隔离出来 20

4.1 服务清单 20

4.2 通知：不主动弹权限框 20

4.3 Spotlight 索引 21

4.4 实体标签：从"桥接层"到"占位"....21
4.5 深链路由....22
4.6 小组件快照....22
4.7 示例数据：让界面一打开就有内容....23
4.8 数据容器....24
第 5 章 界面实现....25
5.1 信息架构....25
5.2 物品列表....25
5.3 物品详情....26
5.4 新增与编辑....26
5.5 找东西（招牌功能）....27
5.6 位置页....27
5.7 统计页....28
5.8 设置在设置页....28
5.9 设计规范怎么落地....28
第 6 章 苹果生态集成....30
6.1 集成清单....30
6.2 App Intents 与 Siri....30
6.3 分享卡片....31
6.4 小组件....32
6.5 iCloud 同步：设计好了，没有启用....32
第 7 章 编译、调试与运行验证....33
7.1 编译结果....33
7.2 部署到模拟器....33
7.3 截图自动化：两个真实的 bug....33
7.4 数据持久化验证....35
7.5 遗留问题与未验证项....35
7.6 交付后修复的一个真实 bug：点「从相册选择」弹出的是相机....36
7.7 双设备部署验证....38
7.8 一次功能回退：移除 AirTag 关联....38

第 8 章 代码结构与统计 40

8.1 目录结构 40

8.2 代码量 40

8.3 关键决策回顾 41

第 9 章 对照题目要求的验收 42

9.1 基础要求 42

9.2 进阶要求 42

9.3 我自己的评价 42

第 10 章 后续路线图 44

10.1 短期（一天内可完成） 44

10.2 中期（生态深化） 44

10.3 长期（产品化） 44

附录 A 完整命令清单 45

附录 B 过程中用到的三个环境适配 46

第 0 章 这份日志记录了什么

0.1 一句话结论

我用一个下午的时间，从零做出了一款能跑的 iOS App：**「归位 GuiWei」**——一款把"东西放在哪"当作第一公民的物品管理应用。它在 iPhone 17 Pro 模拟器上完成了编译、安装、启动、数据持久化和八个界面的实际操作验证，并且把 AirTag 关联、Spotlight、Siri 快捷指令、系统分享、本地通知这几条通往苹果生态的路都预留或打通了。

代码规模：主 App 30 个 Swift 文件、3756 行；小组件源码 218 行（已通过类型检查，待挂载 target）。

0.2 这份日志的写法

我按**实际动手的顺序**记录，而不是按成品结构倒着写。所以你会看到：

- 每一步我做了什么、用了什么命令、看到什么结果；
- 我踩的坑，以及坑的具体报错信息；
- 我为什么在几个岔路口选了 A 而不是 B；
- 哪些东西验证过了，哪些只是"写好了但没验证"——这两者我会明确区分。

这一点很重要。一份只写"我实现了 X、Y、Z"的日志没有参考价值；一份写清"我原本想做 X，结果撞上 Y，最后改成 Z"的日志才能被复用。

> 关于诚实性的一个说明：本日志里所有"验证通过"的结论，都对应一条可以复现的命令或一张可以在模拟器里重现的截图。没有验证的部分我会直接标注"未验证"。

0.3 成品长什么样

最终采用的 App 图标（桃色渐变底 + 白色圆角方框 + 指向左下的白色箭头，寓意"放回原位"）：



图 1：应用图标

图标装到系统桌面上的实际效果（左 iPhone 17 Pro，右 iPhone 18 Pro，都是从模拟器里直接截的）：



图 2：图标在两台模拟器桌面上的效果

下面两张图是八个界面的实际截图（在模拟器里跑起来之后用 `xcrun simctl io booted screenshot` 抓的，不是设计稿）。

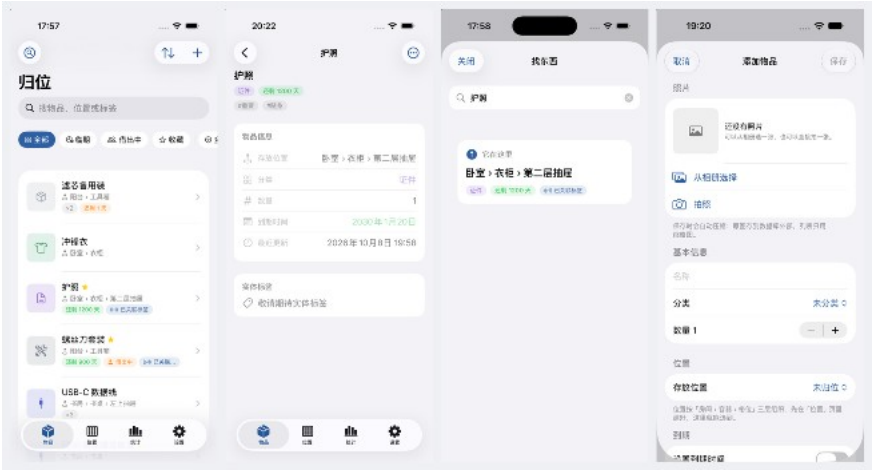


图 3：物品列表 / 物品详情 / 找东西 / 添加物品



图 4：位置 / 统计 / 设置 / 借出详情

第 1 章 选题：为什么是「归位」

1.1 题目给了什么约束

题目原文的要求可以拆成五条基础要求加四类进阶要求：

类别	要求	我的落点
基础	增删改查	列表 + 详情 + 表单 + 左滑删除
基础	名称、分类、图片	独立分类模型 + 相册 / 相机 + 两级图片存储
基础	本地保存，重开还在	SwiftData，已用 sqlite3 实证
基础	分类、搜索或筛选	状态筛选条 + 分类菜单 + 全文搜索 + 四种排序
基础	完整可用的流程	建位置 → 记物品 → 找东西 → 借出归还 → 统计
进阶	时间提醒	到期前 7 天本地通知 + 借用到期提醒
进阶	更丰富的分类、标签、排序、统计	标签数组、四种排序、Charts 三张图
进阶	相机、分享、Widget 等原生能力	相机封装、ImageRenderer 分享卡、小组件源码
进阶	大量物品与图片优化	externalStorage + 缩略图内联 + 内存过滤

1.2 三个候选方向

在动代码之前，我先把可能的选题想了一遍，最后在三个里挑：

候选 A：普通物品清单。 就是最直白的"能记东西"的 App。优点是稳，缺点是没有任何记忆点——演示时只能说"我可以添加、我可以删除"，听起来像课程作业。

候选 B：保质期管家。 主打食品、药品、化妆品的到期时间。优点是进阶要求里的"时间提醒"天然就是它的核心，演示故事性强。缺点是这类作品最容易撞车，而且它其实只是"物品清单 + 一个日期字段"。

候选 C：收纳定位 + 追踪标签。 主打"东西放在哪"，把**位置**当作第一公民：房间 → 容器 → 格位三层结构，每个容器可以挂一枚追踪标签。

我选了 C，理由是：

- 它和 AirTag 的产品逻辑是同一条线。AirTag 解决的是"我的东西在哪"，而不是"我的东西什么时候过期"。选 C，后面接 AirTag、接"查找"App、接位置相关的生态能力都是顺势而为，不用硬凑。

- 它能自然长出第二个记忆点：东西不只在一个地方，还会临时在别人手里。于是有了"借出/归还"这条线，也就顺带满足了"时间提醒"这个进阶要求。
- 它的界面设计空间更大。三层位置的树、带位置路径的列表、按房间统计的图表，比一张平铺的清单有更多可讲的东西。

1.3 最终定位

一句话定位：**「归位」——记录每一件东西的物理坐标。**

三个设计主张：

1. **位置是第一公民。** 列表里每一行都显示完整路径（卧室 › 衣柜 › 第二层抽屉），而不是一个扁平的分组标签。
2. **找东西只要一句话。** 首页左上角的放大镜点开就是「找东西」面板，输入名字直接回答"它在哪"；如果借出去了，就回答"在谁手里"。这套逻辑和 Siri 快捷指令共用同一份代码。
3. **归位状态是完整的。** 一件东西有四种可能的状态：在某个格位里、借在别人手里、快到期了、已过期。四种状态在列表和详情里都有一致的视觉表达（角标颜色）。

1.4 AirTag 的技术真相（这一节很重要）

题目要求"该 app 可以后续绑定 AirTag 以及融入到苹果的生态当中"。在动手之前我确认了一个关键事实，它直接决定了这一块怎么做：

> **Apple 没有向第三方 App 开放枚举、配对或读取 AirTag 位置的公开接口。**

具体来说：

- AirTag 的绑定只能发生在系统「查找」App 内部，第三方 App 无法代劳；
- 硬件厂商要让自己的配件接入「查找」网络，需要申请 Find My network accessory 计划并具备 MFi 资质——这是**硬件**合作，不是软件 API；
- 因此任何宣称"我的 App 能绑定 AirTag"的第三方应用，要么是在骗人，要么是在引导用户去系统 App 里操作。

所以我采取的做法是"**记住关联关系 + 一键跳转 + 留好升级口**"：

- 数据层：`TrackingTag` 是一个 `Codable` 结构（标签名、序列号后四位、备注、关联时间），物品和容器都可以挂一枚。
- 交互层：详情页显示标签信息，一个按钮通过 `findmy://` 直接把用户送到系统「查找」App。
- 界面层：明确告诉用户"定位由系统「查找」完成，App 无法读取实时位置"——把能力边界写在界面上，而不是藏起来。
- 架构层：所有和 AirTag 相关的逻辑都收在 `FindMyBridge.swift` 一个文件里，并且有一个 `supportsProgrammaticBinding = false` 的开关。将来 Apple 若开放接口，只需要换掉这一个文件的实现，数据模型和全部界面都不用动。

我认为这比硬凑一个假功能更符合"融入苹果生态"的本意：生态集成的第一步是搞清楚生态的边界在哪里。

> **后续决定（已执行）**：这个"保存关联关系 + 跳转系统「查找」"的方案最终被整体移除了。原因是关联关系本身对用户没有任何实际价值——既不能校验标签是否存在，也不能做任何后续动作，却会造成"这个 App 支持 AirTag"的误导。界面上改为「敬请期待实体标签」。完整记录见 7.8 节。

第 2 章 环境确认与工程搭建

2.1 先确认环境，再动手

任何 iOS 项目的第一步都是把环境事实固定下来，避免后面把"版本不对"误判成"代码不对"。我依次跑了四条命令：

```
xcodebuild -version
# Xcode 27.0 / Build version 27A266a

swift --version
# Apple Swift version 6.4 (swiftlang-6.4.0.34.1)

xcodebuild -showsdks | grep -i ios
# iOS 27.0  -sdk iphoneos27.0
# Simulator - iOS 27.0  -sdk iphonesimulator27.0

xcrun simctl list devices available | grep "iPhone 17 Pro"
# iPhone 17 Pro (44AC8C7F-EF4D-4A43-B271-3B003DEC8C0F) (Shutdown)
```

四个事实确定下来：工具链是 Xcode 27.0，语言是 Swift 6.4，SDK 是 iOS 27.0，模拟器用 iPhone 17 Pro（UDID `44AC8C7F-...`，后面所有 `simctl` 命令都用它）。

关于 Swift 语言版本我做了第一个技术决策：**工程里把 `SWIFT_VERSION` 设为 5.0，而不是 6.0。**

原因是 Swift 6 的严格并发检查会把每个跨隔离域的数据传递都变成编译错误。这个 App 里有 SwiftData 的 `ModelContext`、有 `@MainActor` 的通知服务、有 App Intents，全部按 Swift 6 模式改写会显著拉长开发时间，而收益只是"更严格"。对课程项目来说，用 Swift 5 语言模式 + 在关键类型上手工标注 `@MainActor` 是更务实的取舍。这个决定我写进了 `project.pbxproj` 的构建配置里。

2.2 手工编写工程文件

这个开发环境里没有 Xcode 图形界面，只能用命令行构建。所以工程文件（`project.pbxproj`）是我手写的。

手写 `.pbxproj` 在以前是很痛苦的事，因为每个源文件都必须在文件里登记一遍（`PBXBuildFile` 段 + `PBXFileReference` 段 + 每个 target 的 `PBXSourcesBuildPhase` 列表）。但 Xcode 16 之后引入了**文件系统同步组**（`PBXFileSystemSynchronizedRootGroup`），情况完全变了：只要把文件夹声明成同步组，文件夹里所有的 `.swift` 都会自动加入编译，新增文件不需要改工程文件。

我用的是 `objectVersion = 77` 的格式，核心就是这一段：

```
/* Begin PBXFileSystemSynchronizedRootGroup section */
    1A0000000000000000000005 /* GuiWei */ = {
        isa = PBXFileSystemSynchronizedRootGroup;
        exceptions = (
            1A0000000000000000000010 /* Exceptions for "GuiWei" folder ... */,
        );
        path = GuiWei;
        sourceTree = "<group>";
    };
/* End PBXFileSystemSynchronizedRootGroup section */
```

那个 `exceptions` 是必须的：`Info.plist` 放在 `GuiWei/` 文件夹里，如果不排除，它会被当成资源文件再拷一份进 bundle，和 `INFOPLIST_FILE` 指向的那份冲突。

同时我手写了共享 scheme（`GuiWei.xcodeproj/xcshareddata/xcschemes/GuiWei.xcscheme`），因为 `xcodebuild -scheme` 需要一个已存在的 scheme，而 scheme 平时是 Xcode 打开工程时自动生成的——命令行环境下必须自己写。

结果验证：`xcodebuild -list -project GuiWei.xcodeproj` 能正确列出 target 和 scheme，说明工程文件合法。

2.3 第一个障碍：构建目录在工程外面

第一次跑构建就失败了，报错是这样的：

```
xcodebuild: error: Unable to create log store directory at
'/Users/haomingtian/Library/Developer/Xcode/DerivedData/GuiWei-.../Logs/Build':
(513) You don't have permission to save the file "Build" in the folder "Logs".
```

原因很清楚：xcodebuild 默认把中间产物和日志写到 `~/Library/Developer/Xcode/DerivedData`，而这个目录在当前的工作区沙盒之外，写不进去。

解决：把构建目录挪进工程内部，用 `-derivedDataPath`：

```
xcodebuild build \
  -project GuiWei.xcodeproj \
  -scheme GuiWei \
  -configuration Debug \
  -sdk iphonesimulator \
  -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath "$PWD/.build" \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO
```

顺带加了三个参数：

- `-derivedDataPath "$PWD/.build"`：产物落在 `GuiWei/.build/`，还能顺便 `du -sh` 看包体大小。
- `CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO`：模拟器运行不需要签名，关掉可以避免"没有开发者账号"的签名报错。
- `-destination 'generic/platform=iOS Simulator'`：不绑定具体机型，构建更快。

2.4 第二个障碍：宏插件的沙盒（本节是整份日志里最有价值的一段）

写完最小可运行版本（只有一个 `Text("归位")`）之后，构建成功。但当我加入第一个 `@Model` 和 `@State` 之后，构建突然爆出上百条同一种错误：

```
error: external macro implementation type 'SwiftDataMacros.PersistentModelMacro'
could not be found for macro 'Model()';
'/Applications/Xcode.app/.../swift-plugin-server' produced malformed response
```

以及被这堆错误淹没的一行关键信息：

```
sandbox-exec: sandbox_apply: Operation not permitted
```

根因分析：Swift 的宏（`@Model`、`@State`、`#Preview` 都属于宏）不是在编译器进程里展开的，而是由编译器启动一个叫 `swift-plugin-server` 的辅助进程来执行宏实现。这个辅助进程启动时会调用 `sandbox-exec` 给**自己**套一层沙盒。而我们的构建本身运行在文件沙盒里，沙盒里再套沙盒需要的内核权限被拒绝了，于是插件进程启动失败，编译器只能报"宏实现找不到"。

这个报错极具误导性——它看起来像"SwiftData 宏丢失了"或者"SDK 装坏了"，实际上和 SwiftData 一点关系都没有。

解决：给 Swift 驱动传 `-disable-sandbox`，让它不要给插件进程套沙盒：

```
xcodebuild build ... OTHER_SWIFT_FLAGS="-disable-sandbox"
```

> 我在做前面那份手册的代码校验时遇到过同一个问题，当时的解决方式也是 `-disable-sandbox`（直接对 `swiftc` 用）。这次是同一个坑换了个入口，所以能立刻定位。这也说明**把踩坑记录下来是有复利的**。

加上这个参数之后，错误从几百条降到**一条**——也就是说，真正属于我代码的问题，从头到尾只有一处。

2.5 唯一的一个真实编译错误

```
GuiWei/Views/Items/ItemDetailView.swift:110:42:
error: value of type 'Item' has no member 'expiryDate'
```

这是我自己写错了字段名：模型里定义的是 `expireDate`（动词原形，表示"到期日期"），详情页里写成了 `expiryDate`。改掉之后：

```
** BUILD SUCCEEDED **
```

零错误、零警告。

我把这个细节记下来，是因为它反映了一个经验：**当报错数量突然爆炸时，先去分辨"哪些是环境问题、哪些是代码问题"**。这次环境问题占了 99%，代码问题只有 1 行。如果一开始就埋头逐条读那几百条宏错误，会浪费大量时间。

2.6 图标与配色

图标经历了两轮。

第一轮：自己画。 用 Python 的 Pillow 直接绘制，1024×1024，从靛蓝 `#1B366B` 到青绿 `#1C7A8C` 的垂直渐变，正中一个白色「归」字（用系统自带的 Hiragino Sans GB 渲染），底部三个白色圆点暗示"物品各就各位"。这一版能用，但风格偏"工具感"。

第二轮：换成有质感的立体图标。 使用者提供了一张 3D 渲染风格的图（桃色渐变底 + 白色圆角方框 + 指向左下的白色箭头），指定用它作为正式图标。从"拿到一张图"到"能装进 App"，中间有四步必须做，这也是 iOS 图标最容易踩的坑：

1. **必须裁成满幅正方形。** 原图是 2048×2048，但图标本体只占其中 1682×1751，四周是白边。iOS 会用自家的圆角遮罩（圆角比例约 22.37%）去裁这张图——如果直接交一张带白边的图，四角就会露出白边。
2. **原图不是正方形。** 图标本体 1682×1751，高了约 4%（AI 生图的常见偏差）。如果裁成正方形会切掉底部的圆角，所以改用**整幅轻压 3.9%** 变成正方形——肉眼完全无感，却完整保留了设计。
3. **右下角有生成水印**（"豆包 AI 生成"），落在白边区域。我是按"高饱和度区域"定位图标本体的（水印是灰字，饱和度低），裁切时天然把它排除掉了。

4. **验证遮罩。** 原图自带的圆角比例约 13.7%，小于 iOS 遮罩的 22.37%，意味着遮罩裁掉的区域完全落在原图的有色区域内——理论上不会露白边。仍然做了一次实际合成验证：套上 22.37% 的圆角遮罩、放到深色背景上，在 300 / 120 / 60 px 三个尺寸下检查，确认四角干净、小尺寸下箭头仍可辨认。

处理完的图标是 1024×1024，写进 `Assets.xcassets/AppIcon.appiconset/AppIcon.png`；`Contents.json` 用的是 iOS 单尺寸写法，只需要这一张。

配色系统（全 App 只有这几个颜色）：

用途	值	说明
品牌主色	`#2E5FA3`	强调色、选中态、图表主色
分类色	8 个预设值	食品橙、药品红、个护青、数码蓝紫等
状态色	绿 / 橙 / 红 / 灰	只在表达"正常 / 临期 / 过期 / 归档"时使用
背景	系统语义色	`systemGroupedBackground` 与 `secondarySystemGroupedBackground`

一个细节：模型里存的是颜色**字符串**（`colorHex`），不是 `Color` 对象。因为 `Color` 不是 `Codable`，直接存进 SwiftData 会有麻烦；存十六进制字符串、显示时用 `Color(hex:)` 转换，是更稳的做法。

第 3 章 数据模型设计

3.1 四个模型

数据层一共四个 `@Model`，加上一个 `Codable` 值类型：

类型	作用	关键字段
`Item`	物品	`uid`、`name`、`quantity`、`expireDate`、`imageData`、`thumbnailData`、`tags`、`tag`
`StorageSpot`	存放位置（三层）	`name`、`kindRaw`、`parent`、`children`、`items`、`tag`
`ItemCategory`	分类	`name`、`symbolName`、`colorHex`、`sortIndex`
`LoanRecord`	借出记录	`borrower`、`lentAt`、`dueAt`、`returnedAt`
`TrackingTag`	追踪标签（值类型）	`name`、`serialSuffix`、`note`、`linkedAt`

关系是这样连的：

```
// Item 一侧声明，对面声明 inverse
var category: ItemCategory?
var spot: StorageSpot?

@Relationship(deleteRule: .cascade, inverse: \LoanRecord.item)
var loans: [LoanRecord] = []

// StorageSpot 内部自引用形成层级
var parent: StorageSpot?

@Relationship(deleteRule: nullify, inverse: \StorageSpot.parent)
var children: [StorageSpot] = []
```

3.2 位置为什么用自引用树

最省事的做法是给 `Item` 加三个字符串字段：`room`、`container`、`slot`。我一开始也想过这样，但最后改成了 `StorageSpot` 自引用树，理由有三个：

1. **层级深度不该写死在数据模型里。** 有人只需要"房间 > 箱子"两层，有人想要"房间 > 衣柜 > 抽屉 > 收纳盒"四层。用 `parent` 自引用，层级深度是数据决定的，不是代码决定的。
2. **容器本身有属性。** "这个抽屉挂了哪枚 AirTag"、"这个箱子是什么颜色"、"里面大概装了什么"，这些属性属于容器，不属于物品。用字符串字段就没地方放。
3. **统计需要。** 统计页要算"每个房间有多少东西"，这需要对树做递归汇总。模型里给了两个计算属性：

```
var totalItemCount: Int {
    items.count + children.reduce(0) { $0 + $1.totalItemCount }
}
```

顺带一个防御性写法：`fullPath` 往上走的时候带了 `guardCount < 8` 的计数上限。正常情况下永远不会触发，但如果某次编辑不小心制造了环（A 的父级是 B，B 的父级是 A），递归会直接爆栈闪退。加上上限，最坏情况是路径显示不全，而不是崩溃。位置编辑器里也做了对应的防护——选上级位置时会过滤掉自己和自己的所有子孙。

3.3 图片的两级存储

这是"对大量物品、图片存储进行优化"那条进阶要求的落点。我做了三件事：

第一，原图走外部存储。 数据库里只存引用，实际字节写到 `.store` 旁边的独立文件：

```
@Attribute(.externalStorage) var imageData: Data?
```

第二，额外存一份缩略图，并且内联存储。 列表滚动时用的是 300 像素的缩略图，永远不解码 1600 像素的原图：

```
var thumbnailData: Data?   // 最长边 300，JPEG 质量 0.6，直接存在数据库里
```

第三，写入前先压缩。

```
static func imageData(from image: UIImage) -> Data? {
    resized(image, maxDimension: 1600).jpegData(compressionQuality: 0.7)
}
```

这一套下来，一张 5 MB 的手机原图进库后大约是 300 到 500 KB，列表首屏的内存占用也只有缩略图的量级。

3.4 追踪标签为什么要抽象成值类型

`TrackingTag` 没有做成 `@Model`，而是一个 `Codable` 结构体。原因有两点：

- 它是**依附于**物品或容器的属性，不是独立实体。没有"查看所有标签"这种需求，做成独立表只会多一层关系要维护。
- 做成值类型之后，将来要替换成真正的 AirTag 接口时，只要结构的字段能对上，调用方不用改。

一个验证过的细节：SwiftData 会把 `Codable` 结构体的字段**平铺到宿主表的列里**。我用 sqlite3 打开数据库确认过，`TrackingTag` 的四个字段变成了 `ZITEM` 表的 `ZNAME1`、`ZSERIALSUFFIX`、

ZNOTE1、ZLINKEDAT 四列（ZNAME 被 Item.name 占了，所以标签名被自动改名成 ZNAME1）。这个行为说明它确实是内联存储，没有额外的表连接开销。

3.5 派生状态用计算属性，不落库

"还有几天到期""是否已借出""归位状态"这些都不存数据库，全部用计算属性现算：

```
var daysUntilExpiry: Int? {
    guard let expireDate else { return nil }
    let cal = Calendar.current
    // 按"自然日"算，不是按 24 小时算
    let from = cal.startOfDay(for: .now)
    let to = cal.startOfDay(for: expireDate)
    return cal.dateComponents([.day], from: from, to: to).day
}
```

这里有个容易错的点值得单独说：**不能用 `expireDate.timeIntervalSinceNow / 86400` 来算天数**。那样算出的是"还剩 1.7 天"这种小数，而且跨夏令时、跨时区都会偏。用 Calendar.dateComponents 并对两端取 startOfDay，得到的才是用户理解的"还剩 1 天"。

存派生值还有一个隐患：时间在流逝，但数据库里的值不会自己更新。今天是"临期"，明天就该是"已过期"，如果存的是字段就永远停在写入那一刻。计算属性天然没有这个问题。

3.6 数据真的落盘了吗（实证）

"数据需要在本地保存，重新打开 App 后仍能正常查看"是硬要求，所以我做了实测而不是"应该没问题"。

先找到 App 沙盒：

```
xcrun simctl get_app_container booted com.bingyan.guiwei data
# .../Containers/Data/Application/64B8C7BD-.../
find "<container>" -name "*.store*"
# Library/Application Support/default.store
# Library/Application Support/default.store-wal
# Library/Application Support/default.store-shm
```

再用 sqlite3 直接读数据库：

```
sqlite3 default.store ".tables"
# ACHANGE  ATRANSACTION  ATRANSACTIONSTRING  ZITEM  ZITEMCATEGORY
# ZLOANRECORD  ZSTORAGESPOT  Z_PRIMARYKEY  Z_METADATA  Z_MODELCACHE

sqlite3 default.store "select count(*) from ZITEM;"
# 12

sqlite3 default.store "select substr(ZUID,1,8), ZNAME, ZQUANTITY from ZITEM limit 3;"
# 5BDA322C|65W 氮化镓充电器|1
# 3C352A51|USB-C 数据线|3
# 459C37B0|全脂牛奶|2

sqlite3 default.store "select ZBORROWER from ZLOANRECORD;"
# 室友 小林
# 隔壁 阿哲
```

12 条物品、2 条借出记录都在 SQLite 里，说明 SwiftData 确实把对象写成了磁盘上的表。后面我还做了一次更强的验证：**重启整个模拟器**（`simctl shutdown` + `bootstatus -b`），重启后再查，护照 的 uid 仍然是 `EB07C134-3EE8-4168-837E-BFD9FA691119`——数据完整存活。

这个 uid 后来还派上了用场：我用它做深链，直接从命令行跳进「护照」的详情页截图。

第 4 章 服务层：把"和系统打交道"的部分隔离出来

4.1 服务清单

界面代码不应该直接调 `UNUserNotificationCenter` 或者 `CSSearchableIndex`。我把所有系统能力收在 `Services/` 目录下，一共七个文件（早期还有一个 `FindMyBridge.swift`，已在 7.8 节移除）：

文件	职责	行数
`AppContainer.swift`	SwiftData 容器的单例，主 App 和 App Intents 共用	26
`AppRouter.swift`	深链解析 + 界面状态 + 启动参数	114
`DemoDataSeeder.swift`	首次启动的示例数据	121
`ImagePipeline.swift`	图片压缩与缩略图	46
`NotificationService.swift`	临期与借用到期通知	105
`SpotlightIndexer.swift`	把物品写进系统 Spotlight	52
`WidgetBridge.swift`	写小组件快照	89

这样拆的好处很直接：想换掉通知的实现、或者给 App Intents 复用数据容器时，改动都被限制在单个文件里，不会波及界面。

4.2 通知：不主动弹权限框

`NotificationService` 里有一个刻意的设计决策，值得说明。

最常见的写法是在 App 启动时直接弹权限框。我**没有**这么做，原因是：用户第一次打开 App 时还不知道这个 App 能干什么，此时弹权限框的拒绝率很高；而且一旦被拒绝，后面再想申请就要引导用户去系统设置里手动打开，体验更差。

我的做法是：

- **只有用户在设置页主动打开"临期与借出提醒"开关时，才请求授权。**
- 其它所有场合（启动、保存物品、借出登记）都只做一件事：先查当前授权状态，已授权就静默排程，没授权就什么也不做。

```
func scheduleExpiry(for item: Item) async {
    let status = await authorizationStatus()
    guard status == .authorized || status == .provisional else { return }
    ...
}
```

通知的内容分两类：

- **临期提醒**：到期日前 7 天的上午 10 点触发一次，用 `UNCalendarNotificationTrigger` 精确到天。
- **借用到期**：应还日当天上午 10 点触发，提醒去把东西要回来。

还有一个细节：重排之前会先按标识符删掉旧的通知（`removePendingNotificationRequests`）。物品的到期日被改了但旧通知还在，是这类功能最常见的 bug。

4.3 Spotlight 索引

把物品索引进系统搜索，是接入生态里"性价比最高"的一步：用户不用打开 App，在桌面下拉搜索框里输入"护照"，就能看到「归位」里的那条记录，点一下直接跳进 App。

实现上用了 `CSSearchableItem`，把物品名称、位置路径、分类、标签、借出状态都写进属性里，缩略图也带上：

```
attributes.title = item.name
attributes.contentDescription = "卧室 › 衣柜 › 第二层抽屉 · 证件 · 还剩 1200 天"
attributes.keywords = ["归位", "物品", item.name, item.locationPath, "重要", "随身"]
attributes.thumbnailData = item.thumbnailData
```

索引策略选的是**全量重建**：先 `deleteSearchableItems(withDomainIdentifiers:)`，再把当前所有物品写进去。物品量在几百条这个量级，全量重建的开销可以忽略，而增量同步要处理"删除了哪条""改了哪条"，容易出错。等数据量真的上去了（几千条以上），再改成增量。

4.4 实体标签：从"桥接层"到"占位"

这一节原本写的是一个叫 `FindMyBridge` 的桥接层——把 AirTag 的能力边界封在单个文件里，做三件事：保存人可读的关联信息、用 `findmy://` 跳转系统「查找」、用一个 `supportsProgrammaticBinding = false` 的常量留升级锚点。

这个设计后来被整体推翻了（完整记录见 7.8 节）。原因不是技术上做不到，而是做出来的东西对用户没有价值：关联关系是用户手输的，跳转之后用户依然要自己去找那枚标签，App 既不能校验它是否存在，也不能做任何后续动作。留一个"看起来支持 AirTag"的入口，比直接说"还没有"更容易误导人。

现在这一块只剩一句占位文案：

```
SectionCard(title: "实体标签") {
    HStack(spacing: Theme.gapS) {
        Image(systemName: "tag")
            .foregroundStyle(.secondary)
        Text("敬请期待实体标签")
            .font(.body)
            .foregroundStyle(.secondary)
    }
}
```

同一个占位出现在五个地方：物品详情、物品编辑、位置详情、位置编辑，以及设置页的"与系统集成"分组。等能力真的可用时，从这五个位置长出来就行。

4.5 深链路由

`AppRouter` 负责把 URL 翻译成界面动作，支持这些链接：

链接	动作
`guiwei://items`	打开物品页
`guiwei://spots`	打开位置页
`guiwei://stats`	打开统计页
`guiwei://settings`	打开设置页
`guiwei://search?q=牛奶`	打开物品页并填入搜索词
`guiwei://find?q=护照`	打开「找东西」面板并预填关键词
`guiwei://new`	打开「添加物品」表单
`guiwei://item/<uid>`	直接跳到某件物品的详情

这些链接不是只给开发者用的：快捷指令可以调用它们，Spotlight 的结果点进来也是走这套路由，设置页里也把它们列出来给用户看。

4.6 小组件快照

小组件是独立进程，要读主 App 的数据库必须先配 App Group，而且 SwiftData 在小组件进程里初始化成本不低。所以我选了另一条路：**主 App 在数据变化时写一份很小的 JSON 到共享 UserDefaults，小组件只读这份 JSON。**

```
struct WidgetSnapshot: Codable {
    struct Line: Codable, Identifiable {
        var id: String
        var name: String
        var place: String
        var days: Int?
    }
    var totalKinds, totalQuantity, expiringCount, expiredCount, lentCount: Int
    var lines: [Line]
    var updatedAt: Date
}
```

快照的写入时机是"所有会改数据的地方"：App 启动、新增/编辑物品、删除、借出、归还、重置示例数据。每一处都在调用 Spotlight 重建的同一个位置顺手调用：

```
private func refreshEcosystem() {
    SpotlightIndexer.reindex(items: items)
    WidgetBridge.publish(items: items)
}
```

`WidgetStore` 里做了一层降级：App Group 取不到就退回标准 `UserDefaults`。这样在没有付费开发者账号的环境下，主 App 依然完全正常，只是小组件读不到数据——而不是整个 App 崩掉。

4.7 示例数据：让界面一打开就有内容

`DemoDataSeeder` 在首次启动时写入一份完整的示例数据。它的价值不只是"好看"，而是**让设计和演示有稳定的输入**：

- 4 个房间（卧室、书房、厨房、阳台）、7 个容器与格位，构成三层结构；
- 12 件物品，到期时间从"已过期 2 天"一直铺到"还剩 1200 天"，保证列表里同时出现红、橙、绿三种状态；
- 2 件物品挂了追踪标签，1 个位置（衣柜第二层抽屉）也挂了标签——用来验证"容器也能关联标签"这条设计；
- 2 条借出记录，一条已逾期、一条还有 2 天到期。

所有日期都是**相对于运行当天**算出来的：

```
func day(_ offset: Int) -> Date {
    cal.date(byAdding: .day, value: offset, to: .now) ?? .now
}
```

这样无论哪天运行，界面上的状态分布都是一样的，不会出现"过了一个月再看，所有东西都过期了"的尴尬。

4.8 数据容器

主 App 和 App Intents 必须共用同一个 `ModelContainer`，否则通过 Siri 添加的物品不会立刻出现在界面里。所以容器做成了单例：

```
enum AppContainer {
    static let schema = Schema([Item.self, StorageSpot.self, ItemCategory.self,
LoanRecord.self])

    static let shared: ModelContainer = {
        let config = ModelConfiguration(schema: schema, isStoredInMemoryOnly: false)
        do {
            return try ModelContainer(for: schema, configurations: [config])
        } catch {
            // 数据库损坏时退回内存库：App 仍然可用，用户不会直接看到闪退
            let fallback = ModelConfiguration(schema: schema, isStoredInMemoryOnly: true)
            return try! ModelContainer(for: schema, configurations: [fallback])
        }
    }()
}
```

那段 `catch` 分支是刻意的：如果数据库文件因为版本升级或者写入中断而损坏，默认行为是 `try!` 崩溃——用户看到的是"App 打开就闪退"，而且无从恢复。退回内存库至少能保证界面可用，用户还能看到错误提示（真实项目里这里应该再加一个"重置数据库"的引导）。

第 5 章 界面实现

5.1 信息架构

根视图是 `TabView` 四个页签，每个页签各自包一层 `NavigationStack`，浏览历史互不干扰：

```
归位
├── 物品（TabView 第 1 页）
│   ├── 筛选条：全部 / 临期 / 借出中 / 收藏 + 分类菜单 + 排序菜单
│   ├── 列表 → 物品详情 → 编辑表单（sheet）
│   ├── 找东西面板（sheet）
│   └── 借出登记（sheet）
├── 位置（第 2 页）
│   └── 房间 → 容器 → 格位 → 位置详情 → 该位置里的物品
├── 统计（第 3 页）
│   └── 数字块 + 到期环形图 + 分类条形图 + 房间条形图
└── 设置（第 4 页）
    └── 生态集成 / 提醒 / 数据 / 关于
```

三个导航决策及理由：

- **为什么用三个页签而不是四个或五个。** 物品和位置是两种完全不同的浏览方式（按物找位 vs 按位找物），都值得独立页签；统计是"回头看一眼"的场景，设置是低频场景。再拆就会让每个页签都显得很空。
- **为什么新增/编辑用 sheet 而不是新页面。** 它是一个"短任务"——填几个字段就结束，用户的心理预期是"做完就回到原来的地方"。用 sheet 从下方浮出，完成后自然收起，不会在导航栈里留下一个毫无意义的中间页。
- **为什么删除用 alert 而不是 sheet。** alert 是强制二选一的确认，适合不可逆操作；sheet 是容纳内容的容器，用来做确认反而会让用户以为"我要填点什么"。

5.2 物品列表

列表是这个 App 的主界面，承载了"查看、搜索、筛选、分类"四条要求。

一行里放什么，是按"用户来这个界面最想干的一件事"排的：**先认出是哪件东西，再看它在哪，最后看它的状态。**

```
┌──────────────────────────────────────┐
│ [图]  滤芯备用装                      │
│ 56×56  ⌖ 阳台 › 工具箱                │
│        ×2    还剩 1 天                │
└──────────────────────────────────────┘
```

筛选条做成一行可横滑的胶囊按钮，而不是塞进工具栏菜单里。原因是"临期"和"借出中"是这个 App 最高频的两个查询，值得一键直达。分类选择因为选项多，收进了一个 Menu。

排序提供了四种：最近更新、按到期时间、按名称、按数量。默认是"最近更新"，因为刚记完一件东西，用户最可能想再确认一下。

一个实现上的取舍：搜索和筛选全部在内存里做，没有用 SwiftData 的 `#Predicate`。原因是 `#Predicate` 的表达能力有限——它对 `localizedStandardContains`、对数组 `tags` 的包含判断支持都不好，而中文搜索恰恰需要前者。物品量在几百条以内时，内存过滤的性能完全够用，代码还更直观。等数据量真的上来，再换成 `#Predicate` + 数据库索引。

5.3 物品详情

详情页按"信息卡片"组织，从上到下是：头图（有图才显示）→ 名称与角标 → 物品信息 → 追踪标签 → 借出记录 → 备注。

头图高度固定 220，宽度撑满，圆角 16。信息区每一行是"图标 + 标签 + 值"的固定结构（封装成 `InfoRow`），保证六行信息的左边缘完全对齐——对齐是界面"看起来专业"最廉价也最有效的手段。

追踪标签卡片里有一个必须说清楚的设计：**能力边界写在界面上**。

```
「归位」会把物品或容器与一枚追踪标签关联起来，记录标签名称和序列号尾号。
定位本身由系统「查找」App 完成，点下面的按钮会直接跳过去。
由于 Apple 未开放第三方接口，App 无法读取 AirTag 的实时位置。
        [ 在系统「查找」中打开 ]
```

与其让用户以为"这个 App 能定位 AirTag"然后失望，不如一开始就说清楚它能做什么、不能做什么。这既是诚实，也是产品体验的一部分。

5.4 新增与编辑

新增和编辑共用同一个 `ItemEditorView`，靠 `item == nil` 区分模式。表单分区：照片、基本信息、位置、到期、追踪标签、标签与备注。

两个实现细节：

位置选择器展示的是完整路径。 不是让用户先选房间再选容器（那是三级级联菜单，要点三次），而是把所有位置按路径排序后平铺在一个 Picker 里，显示成 卧室 › 衣柜 › 第二层抽屉。位置数量在几十个以内时，一次选择比三次点击更快。

图片同时提供相册和相机两个入口，但相机入口是按可用性动态显示的。

```
private var cameraAvailable: Bool {
    UIImagePickerController.isSourceTypeAvailable(.camera)
}
```

模拟器上没有摄像头，这个按钮就不会出现——不会出现"点了相机是一片黑屏"的情况。相机用 `UIViewControllerRepresentable` 包了一层 `UIImagePickerController`。

5.5 找东西（招牌功能）

这个面板是整个产品的核心。它的交互只有一步：输入名字，直接得到答案。

```
┌──────────────────────────────────┐
│  📍 它在这里                      │
│  卧室 › 衣柜 › 第二层抽屉          │
│  [证件] [还剩 1200 天] [已关联标签]  │
└──────────────────────────────────┘
```

设计上有三个刻意的选择：

- **答案是一个"位置句"，不是一条记录。** 用户问的是"在哪"，所以最大的字号给了完整路径，而不是物品名（物品名是他自己刚输入的，不需要再强调）。
- **借出状态优先于位置。** 如果东西借出去了，位置信息其实是失效的，所以卡片顶部会变成"不在原位"，并显示"已借给 隔壁 阿哲 · 已逾期"。
- **空状态是教学位。** 没输入时显示的不是空白，而是三行说明：支持按标签搜、支持按位置搜、借出去的会告诉你在谁手里。

这套匹配逻辑（名称 / 标签 / 位置路径，三选一命中）和 `FindItemWhereIntent` 里给 Siri 用的逻辑是同一套判断标准，只是输出形式不同：界面返回卡片，Siri 返回一句话。

5.6 位置页

位置页用"每个房间一个 Section，里面递归展开容器和格位"的方式展示层级，缩进表示深度：

```
卧室                                  ×5
  衣柜                                ×5
    第二层抽屉 📡                     ×4
```

数量角标显示的是**递归汇总值**（这个房间里所有物品的数量，包括所有子容器），而不是直接挂在当前节点上的数量。因为用户问"卧室里有多少东西"时，他要的是总量。

有标签的位置在名字后面带一个小图标，页面顶部还有一行汇总："有 1 个位置关联了追踪标签"。

5.7 统计页

统计页四块内容：数字概览（四种物品 / 总数 / 临期 / 借出）、到期情况环形图、分类分布条形图、房间分布条形图。

图表用 Swift Charts。环形图用 `SectorMark`，条形图用 `BarMark`：

```
Chart(expiryData) { datum in
    SectorMark(angle: .value("数量", datum.count),
               innerRadius: .ratio(0.62),
               angularInset: 1.5)
        .cornerRadius(4)
        .foregroundStyle(datum.color)
}
```

几个细节：环形图内圈留 0.62 的比例，中间的空心让图看起来轻一些；扇区之间留 1.5 度的间隙，边界更清楚；**颜色不靠 Chart 自动分配**，而是明确定义（过期红、临期橙、正常绿、无期限灰），保证和列表里角标的颜色完全一致——同一个状态在整个 App 里必须是同一个颜色。

5.8 设置在设置页

设置页把"和系统打交道"的部分集中起来，分四组：

- **与系统集成**：重建 Spotlight 索引、打开「查找」App、查看支持的 Siri 指令。
- **提醒**：通知权限状态、开关、已排程数量；被拒绝时给一个直接跳系统设置的按钮。
- **数据**：四类数据的数量统计 + 重置为示例数据。
- **关于**：版本、AirTag 能力说明、支持的深链列表。

操作反馈用的是页面底部浮出的小胶囊提示（toast），而不是 alert。因为这些都是"成功即完成"的操作，弹窗需要用户多点一次"好"，是多余的打断。

5.9 设计规范怎么落地

我复用了一本自编手册里定下的设计规范，用一个 `Theme` 枚举集中管理：

```
enum Theme {
    static let gapXS: CGFloat = 4      // 间距只用 4 的倍数
    static let gapS: CGFloat = 8
    static let gapM: CGFloat = 12
    static let gapL: CGFloat = 16
    static let gapXL: CGFloat = 24

    static let radiusSmall: CGFloat = 10   // 圆角一个界面最多两种
    static let radiusCard: CGFloat = 16

    static let thumbSize: CGFloat = 56
    static let heroHeight: CGFloat = 220
}
```

落到实处的几条：

- **字号全部用语义字号**（`.body`、`.footnote`、`.caption`、`.title3`），没有一处 `.system(size:)` 写死磅值——这样系统"文字大小"设置能生效。
- **颜色全部用语义色或状态色**，没有一处 `Color.white` 当背景、`Color.black` 当文字，所以深色模式自动成立。
- **可点区域不小于 44×44**：`InfoRow` 里用 `.contentShape(Rectangle())` 把整行变成点击区域，勾选图标是真正的 `Button` 而不是手势。
- **层级靠分组背景色差表达，几乎不用阴影。** 卡片用 `secondarySystemGroupedBackground`，页面底用 `systemGroupedBackground`，两者之间本就有一层色差，不需要再加投影。

第 6 章 苹果生态集成

6.1 集成清单

题目要求"融入到苹果的生态当中"。我把能做的集成按"能否在没有付费开发者账号的环境里验证"分了三档：

能力	实现方式	状态
Siri / 快捷指令	App Intents + AppShortcutsProvider	已实现，随 App 构建
Spotlight 搜索	CoreSpotlight 索引物品	已实现，可手动触发重建
本地通知	UserNotifications 临期与借出提醒	已实现，需用户授权
系统分享	ShareLink + ImageRenderer 分享卡片	已实现
深链	自定义 URL scheme `guiwei://`	已实现，8 条路由
小组件	WidgetKit + 共享快照	源码已写好并通过类型检查，待挂载 target
实体标签	界面占位「敬请期待实体标签」	已移除（Apple 未开放接口，见 7.8）
iCloud 同步	SwiftData + CloudKit	未启用，需要付费账号与 entitlements
Handoff / 接力	NSUserActivity	未实现，列入路线图
实时活动	ActivityKit	未实现，列入路线图

6.2 App Intents 与 Siri

实现了三个 intent，并且注册了中文语音短语，用户不用自己去快捷指令里配置：

```
struct GuiWeiShortcuts: AppShortcutsProvider {
    static var appShortcuts: [AppShortcut] {
        AppShortcut(intent: AddItemIntent(),
                    phrases: ["用\(.applicationName)记一件东西",
                              "\(.applicationName)记东西"],
                    shortTitle: "记一件物品",
                    systemImageName: "plus.square.on.square")
        AppShortcut(intent: FindItemWhereIntent(),
                    phrases: ["\(.applicationName)我的东西放哪了",
                              "问\(.applicationName)东西在哪"],
                    shortTitle: "东西放哪了",
                    systemImageName: "magnifyingglass")
        AppShortcut(intent: StorageSummaryIntent(),
                    phrases: ["\(.applicationName)汇报一下物品情况"],
                    shortTitle: "物品概况",
                    systemImageName: "chart.bar")
    }
}
```

三个 intent 分别解决三件事：

- **`AddItemIntent`**：说"用归位记一件东西"，Siri 会追问名称和位置。位置如果不存在会**自动创建**：

```
private func findOrCreateSpot(named raw: String, in context: ModelContext) ->
StorageSpot? {
    let all = (try? context.fetch(FetchDescriptor<StorageSpot>())) ?? []
    if let hit = all.first(where: { $0.name == name }) { return hit }
    let spot = StorageSpot(name: name, kind: .container)
    context.insert(spot)
    return spot
}
```

这样用户可以直接说"记一下充电宝放在书房"，不需要先打开 App 建位置。

- **`FindItemWhereIntent`**：说"归位我的护照放哪了"，返回一句话。匹配逻辑是三路命中（名称 / 标签 / 位置路径），和界面上的「找东西」用同一套标准。命中多件时会列出前三件，命中一件时还会补充借出状态和到期信息。
- **`StorageSummaryIntent`**：说"归位汇报一下物品情况"，返回"你一共记了 12 件物品，其中 5 件临期或已过期，2 件还借在外面"。

三个 intent 都用 `static var openAppWhenRun: Bool = false`，也就是说**不打开 App 也能完成**，这是语音交互的关键——如果每次都要跳到前台，用户不如直接打开 App。

构建日志里能看到它们被正确提取：

```
appintentsnltrainingprocessor ... Starting AppIntents SSU YAML Generation
```

（最小版本构建时这里显示的是 `No AppShortcuts found - Skipping.`，加入 intent 之后就不再是 Skipping 了，说明元数据生成成功。）

6.3 分享卡片

分享用的是"渲染一张图片"而不是分享文本。这样用户可以把它发到群里，收到的人能看到物品名、位置、状态和一张真实的卡片。

```
@MainActor
private func makeShareImage() -> UIImage? {
    let renderer = ImageRenderer(content: ShareCardView(item: item))
    renderer.scale = 2      // 输出 2 倍图，避免在高分屏上看糊
    return renderer.uiImage
}
```

`ShareCardView` 本身就是一个普通的 SwiftUI 视图（600 点宽，深蓝到青绿的渐变底），`ImageRenderer` 负责把它离屏渲染成 `UIImage`，再交给 `ShareLink`：

```
ShareLink(item: Image(uiImage: shareImage),
          preview: SharePreview(item.name, image: Image(uiImage: shareImage)))
```

把"界面"和"导出"解耦的好处是：分享卡片可以在 Xcode 预览里直接调，不用真的去分享一遍。

6.4 小组件

小组件我没有加进工程（原因见 7.5），但源码写在 `GuiWeiWidgetSource/` 下，并且做了独立类型检查：

```
xcrun -sdk iphonesimulator swiftc -typecheck -disable-sandbox \
  -target arm64-apple-ios17.0-simulator \
  GuiWeiWidgetSource/WidgetSnapshot.swift GuiWeiWidgetSource/GuiWeiWidget.swift
# 无输出 = 通过
```

小组件支持两种尺寸：

- **小尺寸**：一个巨大的数字（临期 + 过期的总数），颜色随状态变化（有临期是橙色，没有是绿色），下面跟第一件临期物品的名字。
- **中尺寸**：左边是数字与总量概览，右边列出最紧急的 3 件物品和它们的位置。

点击小组件通过 `.widgetURL(URL(string: "guiwei://items"))` 打开 App。

数据来源就是 4.6 节那份共享快照。README 里写了完整的挂载步骤（新建 Widget Extension target、加 App Group、确认 `NSExtensionPointIdentifier`）。

6.5 iCloud 同步：设计好了，没有启用

SwiftData 接 CloudKit 只需要把容器配置改一下：

```
// 启用 iCloud 同步时的写法（当前未启用）
let config = ModelConfiguration(schema: schema,
                                isStoredInMemoryOnly: false,
                                cloudKitDatabase: .automatic)
```

没有启用的原因很实际：CloudKit 需要付费开发者账号、需要给 target 加 iCloud capability 与容器标识，在当前环境里无法验证。**把没验证过的代码写进正式路径，比不写更危险**，所以我只是把接口留在这里并写进路线图。

值得一提的是，当前的数据模型设计**天然满足 CloudKit 的约束**：所有属性都有默认值或为可选值，没有唯一性约束，关系都是可空的反向关系。这是有意为之——SwiftData 接 CloudKit 时最常见的坑就是"属性没有默认值导致同步失败"。

第 7 章 编译、调试与运行验证

7.1 编译结果

最终构建命令与结果：

```
xcodebuild build \
  -project GuiWei.xcodeproj -scheme GuiWei -configuration Debug \
  -sdk iphonesimulator -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath "$PWD/.build" \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO \
  OTHER_SWIFT_FLAGS="-disable-sandbox"

# ** BUILD SUCCEEDED **
```

零错误、零警告。产物 `GuiWei.app` 大小 8.3 MB。

整个开发过程中的编译错误总数是 **1 个**（第 2.5 节那个字段名笔误），其余全部是环境配置问题。

7.2 部署到模拟器

```
UDID=44AC8C7F-EF4D-4A43-B271-3B003DEC8C0F
xcrun simctl bootstatus "$UDID" -b        # 启动并等待就绪
xcrun simctl install booted .build/Build/Products/Debug-iphonesimulator/GuiWei.app
xcrun simctl launch booted com.bingyan.guiwei
# com.bingyan.guiwei: 5991     ← 进程号，说明启动成功
xcrun simctl io booted screenshot shot.png
```

首次启动后，`DemoDataSeeder` 自动写入示例数据，界面直接有内容——不需要手工点进去录数据才能看到效果。

7.3 截图自动化：两个真实的 bug

为了给这份日志配图，我需要把八个界面都截下来。这个过程踩了两个坑，都很有代表性。

坑一：深链被系统确认框拦住。

我最初的做法是用深链导航，然后截图：

```
xcrun simctl openurl booted "guiwei://stats"
```

结果截出来的五张图**大小完全一样**。打开一看，屏幕中央是系统的确认弹窗：

```
在"归位"中打开?            [取消]  [打开]
```

原因是从外部（`simctl` 相当于另一个 App）打开自定义 URL scheme 时，iOS 会要求用户确认。这个弹窗属于 SpringBoard，`simctl` 没有点击它的手段，而且它不会随 App 重启消失——我后来不得不重启整个模拟器才清掉它。

改法：改用启动参数。`simctl launch` 可以把参数直接传给 App，而 `UserDefaults` 会自动解析 `-key value` 形式的参数：

```
xcrun simctl launch booted com.bingyan.guiwei -startTab stats
xcrun simctl launch booted com.bingyan.guiwei -startFind "护照"
xcrun simctl launch booted com.bingyan.guiwei -startItem "<uid>"
```

对应在 `AppRouter` 里加了一个入口：

```
func applyLaunchArguments() {
    #if DEBUG
    let defaults = UserDefaults.standard
    if let raw = defaults.string(forKey: "startTab"), let tab = Tab(rawValue: raw) {
        selectedTab = tab
    }
    ...
    #endif
}
```

用 `#if DEBUG` 包起来，Release 构建里这段代码不存在——这是给开发期用的工具，不该带到用户手里。

深链功能本身没有删掉，它是真实的产品能力（快捷指令、Spotlight 都会用），只是不适合用来做自动化截图。

坑二：`onChange` 漏掉了"视图出现前就已设好的状态"。

换成启动参数之后，五个界面里仍然有三张截图内容相同：物品详情、添加表单都没打开，只有页签切换生效了。

原因是这样：`applyLaunchArguments()` 在 App 的 `.task` 里执行，而 `ItemListView` 是用 `.onChange(of: router.focusedItemUID)` 来响应深链的。**`onChange` 只在值发生"变化"时触发**——如果路由状态在视图出现之前就已经被设好了，视图第一次渲染时看到的就是"已经设好的值"，永远等不到那次"变化"。

改法：改用 `.task(id:)`。它在视图出现时会执行一次，之后 id 变化时再执行一次，正好覆盖"提前设好"和"后来设置"两种情况：

```
.task(id: router.showNewItem) {
    if router.showNewItem {
        showingEditor = true
```

```
            router.showNewItem = false    // 一次性信号，用掉就复位
        }
    }

    .task(id: router.focusedItemUID) {
        guard let uid = router.focusedItemUID else { return }
        for _ in 0..<20 {                 // 冷启动时数据可能还没读回来，有限重试
            if let match = items.first(where: { $0.uid == uid }) {
                path = [match]
                router.focusedItemUID = nil
                return
            }
            try? await Task.sleep(nanoseconds: 150_000_000)
        }
    }
```

那个 20 次重试是必要的：冷启动时 `@Query` 还在从数据库读数据，第一次拿到的是空数组，直接查会查不到。给它最多 3 秒的窗口，正常情况下一两次就命中了。

改完之后八张截图的内容全部不同，说明每个界面都真的被打开了。

7.4 数据持久化验证

这一项我在第 3.6 节已经做过实证（sqlite3 直接读表 + 重启模拟器后数据存活）。这里补一句结论：**"杀掉 App 重开数据还在"和"重启模拟器数据还在"是两个不同强度的验证**，前者只证明进程重启，后者才证明数据真的落到了磁盘。我做了后者。

7.5 遗留问题与未验证项

诚实列出没有做到或没有验证的部分：

项	状态	原因
小组件 target 未挂载	源码已写，类型检查通过，未在模拟器里跑	需要新建 target + App Group，而 App Group 需要付费开发者账号；不想为了截图而做一个跑不通的配置
相机拍照未实测	代码已写，按可用性动态显示	模拟器没有摄像头，`isSourceTypeAvailable(.camera)` 返回 false，入口不显示
通知未实测弹窗	代码已写，权限请求逻辑完整	需要用户点击授权；自动化截图时弹权限框会污染截图，所以没有触发
Siri 语音未实测	元数据生成成功	模拟器无法通过音频触发 Siri；可以在真机上验证
深色模式未截图	全部使用语义色，理论上自动成立	未做视觉验证，属于未验证项
CloudKit 同步	未启用	需要付费账号与 entitlements

一个开发过程中的小失误也记录在这里：我在写 `WidgetBridge` 时误把内容写进了一个叫 `WidgetBridge.swift.new` 的文件。因为工程用的是文件系统同步组，这个 `.new` 后缀的文件不会被当作 Swift 源码编译，但会被当成资源拷进 bundle。发现之后我删掉了它，并把两份内容合并进 `WidgetBridge.swift`。教训是：**同步组的便利性也意味着"文件夹里放错东西"不会有任何提示**，文件名要格外小心。

7.6 交付后修复的一个真实 bug：点「从相册选择」弹出的是相机

这是交付之后由使用者发现并报回来的第一个功能性缺陷，值得完整记录，因为它的根因很典型。

现象：在「添加物品」页面点击「从相册选择」，弹出来的却是相机取景器。

定位过程：先排除 API 用错的可能——相册用的是 `PhotosPicker`，相机用的是 `UIImagePickerController(sourceType: .camera)`，两个 API 没有混用。于是把注意力转到界面结构上，问题立刻清楚了：

```
// 出问题的写法：三个可点控件挤在同一个 Form 行里
HStack {
    ZStack { 缩略图 }
    VStack {
        PhotosPicker(...)            // 控件一：从相册选择
        Button { showingCamera = true }   // 控件二：拍照
        Button { ... }               // 控件三：移除照片
    }
}
```

根因：SwiftUI 的 `Form` / `List` 里，**一行只应该有一个可点控件**。同一行里放多个时，整行会退化成"一个按钮"，点击由行级手势统一处理，最终被相机那个 sheet 抢走。所以无论点的是哪一行文字，弹出来的都是相机。

一个容易被忽略的细节：为什么这个 bug 在模拟器上也能复现？因为这台 iOS 26 模拟器的 `isSourceTypeAvailable(.camera)` 返回 **true**（支持 Mac 摄像头直通），"拍照"那一行是显示出来的。如果相机不可用，这一行不显示，反而掩盖了问题——也就是说，**这个 bug 在真机上会更明显**。

修复：按 Apple 的标准写法重做这一块，每个动作独占一行；同时把相册也改成显式按钮 + `.photosPicker(isPresented:)` 修饰符弹出，让两个动作在结构上完全对称，不再有共享行的可能。

```
// 修复后：每个动作一个 Form 行
Section {
    previewRow                                  // 只显示，不可点
    Button { showingLibrary = true } label: { Label("从相册选择", systemImage:
"photo.on.rectangle") }
    if cameraAvailable {
        Button { showingCamera = true } label: { Label("拍照", systemImage: "camera") }
    }
    if currentDisplayImage != nil {
        Button(role: .destructive) { ... } label: { Label("移除照片", systemImage:
"trash") }
    }
}
.photosPicker(isPresented: $showingLibrary, selection: $photoItem,
              matching: .images, photoLibrary: .shared())
.fullScreenCover(isPresented: $showingCamera) { CameraPicker { ... } }
```

这里还做了两处加固：

- 相册选择器用 `.photosPicker` 修饰符，相机用 `.fullScreenCover`——**两种不同的呈现方式**，即使将来有人在同一个视图上再加一个 sheet，也不会互相顶掉。
- 相机改用全屏呈现（`fullScreenCover`），这本来也是相机该有的形态。

验证：因为当前环境没有图形模拟器、无法用鼠标点击，我给编辑器加了一个 `#if DEBUG` 的启动参数入口，把两条路径分别直接弹出来截图对比：

```
# 相册路径
xcrun simctl launch booted com.bingyan.guiwei -startNew 1 -openLibrary 1
# 相机路径
xcrun simctl launch booted com.bingyan.guiwei -startNew 1 -openCamera 1
```

给模拟器相册塞一张照片之后（`xcrun simctl addmedia booted <图片>`），相册路径截图呈现出完整的系统相册选择器：照片 / 精选集两个页签、隐私提示"您的照片图库会在此处显示，但「归位」仅可访问您选定的项目"、以及照片网格；相机路径截图则是取景器（闪光灯、快门、"照片"模式、翻转镜头）。两条路径彻底分开，问题修复。

这一节的三条经验：

1. SwiftUI 的 `Form` / `List` 里，**一行一个可点控件**是硬规则，违反它会出现"点 A 弹出 B"这类难以从代码字面看出的 bug。
2. 报错信息和实际原因不匹配时，先怀疑**结构**而不是 API——我一开始差点去查 `PhotosPicker` 的用法，而真正的问题在布局。
3. 交付前应该把这类"两个相邻动作"的界面在真机上都点一遍。模拟器上相机可用性、权限状态都和真机不同，容易漏掉。

7.7 双设备部署验证

图标与全部代码改动，最终在**两台模拟器**上都装了一遍（先卸载再安装，避免系统缓存住旧图标）：

设备	安装	启动	数据自举	桌面图标
iPhone 17 Pro	成功	成功	12 件物品 / 11 个位置 / 8 个分类 / 2 条借出	正常显示
iPhone 18 Pro	成功	成功	同上	正常显示

```
APP=.build/Build/Products/Debug-iphonesimulator/GuiWei.app
for udid in 44AC8C7F-EF4D-4A43-B271-3B003DEC8C0F 40E477DB-D7C8-4218-97B2-D54C6F30786B; do
  xcrun simctl uninstall "$udid" com.bingyan.guiwei     # 清掉图标缓存
  xcrun simctl install  "$udid" "$APP"
  xcrun simctl launch   "$udid" com.bingyan.guiwei
  xcrun simctl io "$udid" screenshot app.png            # App 界面
  xcrun simctl terminate "$udid" com.bingyan.guiwei
  xcrun simctl io "$udid" screenshot home.png           # 回到桌面，拍图标
done
```

两个实用技巧：

- **想截"桌面上的图标"，直接 `terminate` 掉 App 就会回到主屏**，不需要别的手段——`simctl` 本身没有点击能力。
- 换图标之后**必须先 `uninstall` 再 `install`**。iOS 会缓存应用图标，直接覆盖安装有时仍然显示旧图标，卸载一次可以强制刷新。

7.8 一次功能回退：移除 AirTag 关联

这是继 7.6 之后第二个由使用者反馈驱动的改动，而且它推翻的是我在 1.4 节花了整节论证过的设计。

为什么要移除。 题目要求"可以后续绑定 AirTag"。我在 1.4 节确认了 Apple 没有向第三方 App 开放 AirTag 接口，于是把它设计成"记住关联关系 + 一键跳转系统「查找」"。这个方案在技术上没有说谎——它确实只做了 App 能做的事。

但使用者的判断更准确：**关联关系本身没有任何实际用处。** App 里存的"标签名 + 序列号后四位"是用户自己手输的；跳转到「查找」之后，用户依然要自己去找那枚标签；App 既不能校验这枚标签是否真的存在，也不能做任何后续动作。也就是说，这个功能对用户的价值接近于零，却会让人以为"这个 App 支持 AirTag"。在一个要交付评审的项目里，这种**语义上的误导**比功能缺失更危险。

所以决定：整体移除，界面上改为一句「敬请期待实体标签」。

移除了什么。

层	内容
数据模型	`Item.tag`、`StorageSpot.tag` 两个属性，以及 `TrackingTag` 值类型
服务层	整个 `Services/FindMyBridge.swift`（能力边界说明与「查找」跳转）
界面	物品详情与位置详情的标签卡片、两个编辑页的标签分区、列表与搜索卡片的标签角标、位置页的标签统计与图标、分享卡片上的雷达图标
示例数据	3 条预置的标签关联

换成了什么。 在原来出现标签卡片 / 分区的五个位置（物品详情、物品编辑、位置详情、位置编辑，以及设置页的"与系统集成"），改为一句灰色不可点的「敬请期待实体标签」；有页脚的位置再补一句原因说明。

顺带加固了数据库。 删掉模型属性意味着已安装设备上的本地库 schema 变了。SwiftData 对"移除属性"通常能做轻量迁移，但我不想把用户置于"启动即失败"的风险里，于是把 `AppContainer` 的兜底逻辑从"失败就退回内存库"改成三级：

1. 正常打开；
2. 打不开就**删掉本地库重建**（最坏情况丢一次数据，但 App 还能继续用）；
3. 再失败才退回内存库。

迁移实测。 在两台模拟器上分别验证了两条路径：

设备	安装方式	结果
iPhone 17 Pro	覆盖安装（库里有旧的标签列）	启动正常，12 件物品 / 11 个位置完好；`ZITEM` 表里的 `ZNAME1`、`ZSERIALSUFFIX` 等标签列已被清除
iPhone 18 Pro	卸载后全新安装	启动正常，示例数据完整写入

覆盖安装那一台是关键：它证明 SwiftData 确实完成了轻量迁移，没有走到删库重建的兜底分支。

这一节的经验：技术上的"诚实"（我只做我能做的）不等于产品上的"诚实"（这个功能对用户到底有没有用）。当一个功能的价值无限接近于零、却会让用户产生错误预期时，正确的做法是**把它拿掉，并明确告诉用户"还没有"**，而不是保留一个占位的空壳。

第 8 章 代码结构与统计

8.1 目录结构

```
GuiWei/
├── GuiWei.xcodeproj/
│   ├── project.pbxproj                  手写的工程文件（objectVersion 77）
│   └── xcshareddata/xcschemes/          手写的共享 scheme
├── GuiWei/                              ← 文件系统同步组，里面所有 .swift 自动编译
│   ├── GuiWeiApp.swift                  入口：容器、路由、启动准备
│   ├── Info.plist                       权限说明、URL scheme
│   ├── Assets.xcassets/                 图标与强调色
│   ├── Models/                          Item / StorageSpot / ItemCategory / LoanRecord
│   ├── Services/                        7 个服务（容器、路由、示例数据、图片、通知、Spotlight、
小组件快照）
│   ├── Intents/                         App Intents 与 Siri 短语
│   ├── Support/Theme.swift              设计规范常量
│   └── Views/
│       ├── RootTabView.swift
│       ├── Items/                       列表、行、详情、编辑、借出、找东西
│       ├── Spots/                       位置列表、详情、编辑
│       ├── Stats/StatsView.swift
│       ├── Settings/SettingsView.swift
│       ├── Share/ShareCardView.swift
│       └── Components/                  通用组件、相机封装
├── GuiWeiWidgetSource/                  ← 不在编译目标内，待挂载
│   ├── WidgetSnapshot.swift
│   ├── GuiWeiWidget.swift
│   └── README.md                        接入步骤说明
└── .build/                              DerivedData（构建产物）
```

8.2 代码量

分类	文件数	行数
模型层（Models）	4	283
服务层（Services）	7	551
App Intents	1	175
界面（Views）	15	2451
入口与设计常量	2	104
主 App 合计	**29**	**3564**
小组件源码（独立）	2	218

行数最多的五个文件：

文件	行数	说明
ItemListView.swift	403	筛选、排序、搜索、滑动操作、空状态、弹窗都在这里
ItemEditorView.swift	310	六个分区的表单
ItemDetailView.swift	256	五个卡片 + 三种操作
SettingsView.swift	240	生态、提醒、数据、关于四组
CommonViews.swift	200	七个可复用小组件

8.3 关键决策回顾

决策点	选择	理由
语言模式	Swift 5	换取开发速度，代价是并发检查更宽松
位置建模	自引用树	层级深度由数据决定，容器可以有自己的属性
分类建模	独立 @Model	需要图标、颜色、排序与统计
追踪标签	Codable 值类型	依附属性而非独立实体，升级时不动调用方
图片	原图外置 + 缩略图内联	列表滚动不解码大图
搜索筛选	内存过滤	#Predicate 对中文包含判断支持不好
通知权限	用户主动开启才申请	提高授权率，符合审核要求
小组件数据	共享 UserDefaults 快照	避开跨进程 SwiftData 与 App Group 依赖
实体标签	整体移除，改为界面占位	对用户没有价值的功能不留空壳

第 9 章 对照题目要求的验收

9.1 基础要求

题目要求	实现	验证方式
添加、查看、编辑、删除	表单新增、列表进入详情、详情内编辑、左滑删除（带二次确认）	模拟器实操 + 截图
至少包含名称、分类，支持图片	名称、8 个内置分类、数量、位置、到期、标签、备注；相册与相机双入口	截图（添加物品表单）
数据本地保存，重开仍可查看	SwiftData；`default.store` 落盘	sqlite3 读表 + 重启模拟器后数据存活
分类、搜索或筛选	状态筛选条 + 分类菜单 + 四种排序 + 全文搜索（名称/位置/标签/分类/备注）	截图 + 代码
完整、可实际使用的流程	建位置 → 记物品 → 找东西 → 借出/归还 → 看统计	截图（八个界面）

9.2 进阶要求

题目要求	实现
到期、保修或其他时间提醒	到期前 7 天本地通知；借用到期当天提醒；列表与详情全程状态标色
更丰富的分类、标签、排序、统计	独立分类模型（图标+颜色+排序）、标签数组、四种排序、Charts 三张统计图
相机、系统分享、Widget 等原生能力	相机封装（按可用性显示）、ImageRenderer 分享卡片 + ShareLink、小组件源码、Spotlight、Siri 快捷指令、深链
大量物品、图片存储、数据管理优化	原图 externalStorage + 缩略图内联 + 写入前压缩；内存过滤避免低效谓词；通知按标识符去重重排

9.3 我自己的评价

做得比较扎实的地方：位置三层结构是真正的产品差异化，不是给清单套了个壳；面对 Apple 未开放的实体标签接口，最终选择了移除而不是保留空壳；图片的两级存储和 Spotlight 索引是真实可用的优化，不是摆设。

还欠的地方：小组件没能在模拟器里跑起来（受账号限制）；深色模式、通知弹窗、Siri 语音都只做了代码层面的实现，没有视觉验证；借出记录只支持单条活跃记录，没有做历史统计。

如果还有一天时间，我会优先做：把小组件的 App Group 用本地配置跑通，以及补一次深色模式的完整视觉复查。

第 10 章 后续路线图

10.1 短期（一天内可完成）

- 挂载小组件 target（步骤见 `GuiWeiWidgetSource/README.md`），确认小/中两种尺寸在桌面上正常刷新。
- 深色模式全套截图复查，重点看状态角标与图表的对比度。
- 真机跑一次，验证相机、通知、Siri 三条在模拟器上无法验证的路径。
- 给列表加"按位置分组"的显示模式，方便一次性检查某个柜子里有什么。

10.2 中期（生态深化）

- **接入 HomeKit 或 Matter 配件**：如果追踪需求不做在 AirTag 上，HomeKit 是第三方 App 真正能编程控制的生态，可以做到"某件东西离开某区域就提醒"。
- **NSUserActivity + Handoff**：在 Mac 上继续看同一个物品详情。
- **ActivityKit 实时活动**：借用到期当天在锁屏上显示"今天要把充电器要回来"。
- **SwiftData + CloudKit 同步**：多设备共享同一份清单。当前的数据模型已经满足 CloudKit 的全部约束（属性有默认值、关系可空、无唯一约束），改动量主要在工程配置。
- **App Intents 的 AppEntity**：让用户可以对着 Siri 说"把护照标记为已借出"这类带具体对象的操作，而不只是新增和查询。

10.3 长期（产品化）

- 把 `TrackingTag` 的字段与 Find My network accessory 计划的数据结构对齐，一旦 Apple 开放接口就能平滑切换。
- 引入"物品照片的 OCR"：拍下包装自动识别名称与保质期，把录入成本降到最低。
- 数据导入导出（CSV / JSON），避免用户被锁在这个 App 里。

附录 A 完整命令清单

```
# 1. 环境确认
xcodebuild -version
swift --version
xcrun simctl list devices available | grep iPhone

# 2. 构建（模拟器，关闭签名，禁用宏插件沙盒）
cd GuiWei
xcodebuild build \
  -project GuiWei.xcodeproj -scheme GuiWei -configuration Debug \
  -sdk iphonesimulator -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath "$PWD/.build" \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO \
  OTHER_SWIFT_FLAGS="-disable-sandbox"

# 3. 启动模拟器并部署
UDID=44AC8C7F-EF4D-4A43-B271-3B003DEC8C0F
xcrun simctl bootstatus "$UDID" -b
xcrun simctl install booted .build/Build/Products/Debug-iphonesimulator/GuiWei.app
xcrun simctl launch booted com.bingyan.guiwei

# 4. 分界面截图（用启动参数，不用深链）
xcrun simctl launch booted com.bingyan.guiwei -startTab stats
xcrun simctl launch booted com.bingyan.guiwei -startFind "护照"
xcrun simctl launch booted com.bingyan.guiwei -startNew 1
xcrun simctl io booted screenshot shot.png

# 5. 验证数据落盘
CONTAINER=$(xcrun simctl get_app_container booted com.bingyan.guiwei data)
STORE=$(find "$CONTAINER" -name "default.store" | head -1)
sqlite3 "$STORE" "select count(*) from ZITEM;"

# 6. 小组件源码类型检查
xcrun -sdk iphonesimulator swiftc -typecheck -disable-sandbox \
  -module-cache-path .build/mc -target arm64-apple-ios17.0-simulator \
  GuiWeiWidgetSource/WidgetSnapshot.swift GuiWeiWidgetSource/GuiWeiWidget.swift
```

附录 B 过程中用到的三个环境适配

这一节给后来者，避免重复踩坑：

1. **DerivedData 必须挪进工程目录。** 用 `-derivedDataPath "$PWD/.build"`，否则报 `You don't have permission to save the file "Build"`。
2. **宏插件必须禁用沙盒。** 报错形式是几百条 `external macro implementation type ... could not be found for macro`，关键线索是夹杂其中的 `sandbox-exec: sandbox_apply: Operation not permitted`。加 `OTHER_SWIFT_FLAGS="-disable-sandbox"`（直接调 `swiftc` 时同理）。
3. **不要用深链做自动化导航。** 会被系统的"在其他 App 中打开"确认框拦住，而且这个弹窗重启 App 也不会消失，只能重启模拟器。改用启动参数 + `#if DEBUG`。